



UNIVERSIDAD DEL VALLE
ESCUELA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Calculando el Plan de Riego Óptimo de una Finca

Autores:

Ervin Caravali Ibarra

`ervin.caravali@correounivalle.edu.co`

Camilo Urrea

`camilo.urrea@correounivalle.edu.co`

Asignatura:

Análisis de Algoritmos II

Cali, Colombia
2026

ÍNDICE

I.	Introducción y Contexto	2
I-A.	El Problema Real y la Importancia del Riego Óptimo	2
I-B.	Restricciones Prácticas y Estrés Hídrico	2
I-C.	Motivación Tecnológica e Impacto	2
II.	Formalización Matemática	2
II-A.	Definición del Espacio del Problema: La Finca	2
II-B.	La Programación de Riego (Permutación)	2
II-C.	Cálculo de Tiempos y Función de Costo Individual (CRF)	2
II-D.	Función de Costo Objetivo	3
III.	Naturaleza Combinatoria del Problema	3
IV.	Análisis Completo de Ejemplos y Funcionamiento Interno	3
IV-A.	Fuerza Bruta: El Árbol de Búsqueda (DFS)	3
IV-B.	Voraz: Matriz de Descarte Secuencial y Método Piloto	3
IV-C.	Programación Dinámica: Matriz de Ensamblaje Bottom-Up	3
V.	Validación Empírica	3
VI.	Algoritmo 1 — Fuerza Bruta	3
VI-A.	Idea Conceptual	3
VI-B.	Complejidad Computacional (Fuerza Bruta)	3
VI-C.	Consistencia y Desempate Lexicográfico	5
VI-D.	Demostración Matemática (Fuerza Bruta)	5
VII.	Algoritmo 2 — Voraz	5
VII-A.	Heurística WMDD y Método Piloto	5
VII-B.	Complejidad Computacional (Voraz con WMDD)	5
VII-C.	Limitaciones Teóricas	6
VII-D.	Demostración Matemática (Algoritmo Voraz)	6
VIII.	Algoritmo 3 — Programación Dinámica	6
VIII-A.	Subestructura Óptima y Memoización	6
VIII-B.	Complejidad Computacional (Programación Dinámica)	6
VIII-C.	Demostración Matemática (Programación Dinámica)	7
IX.	Diseño de Casos de Prueba	7
X.	Comparación Global de los Algoritmos	7
XI.	Detalles de Implementación	8
XII.	Análisis de Correctitud	8
XIII.	Resultados Experimentales	8
XIII-A.	Análisis Detallado del Comportamiento por Algoritmo	8
XIV.	Discusión Académica	8
XV.	Trabajos Futuros	8
XVI.	Conclusiones	8
XVII.	Referencias	8
	Referencias	8

Resumen—Este trabajo compara tres estrategias para calcular el orden de riego que minimice las pérdidas por estrés hídrico en una finca cañera: Fuerza Bruta, un algoritmo Voraz con lookahead predictivo y Programación Dinámica con memoización. Mostramos que el problema pertenece a la clase NP-Hard, lo que hace inviable la Fuerza Bruta para instancias grandes. La Programación Dinámica reduce la complejidad de $O(n!)$ a $O(n \cdot 2^n)$ y produce resultados exactos hasta $n \approx 18$, mientras el Voraz escala sin problemas hasta $n = 25$ con errores menores al 3%. Los tres algoritmos se validaron contra 19 casos de prueba con salidas esperadas conocidas. También se discute cómo integrar este motor algorítmico en una arquitectura web real con frontend en React/Angular, backend en Node.js y lógica de optimización en Python.

Palabras clave—Programación Dinámica, Algoritmos Voraces, Optimización Combinatoria, Scheduling, Riego Agrícola, NP-Hard.

I. INTRODUCCIÓN Y CONTEXTO

I-A. El Problema Real y la Importancia del Riego Óptimo

La agricultura moderna enfrenta el desafío de maximizar la producción mientras reduce el consumo de recursos finitos, particularmente el agua. En regiones de alta intensidad agrícola como el Valle del Cauca, la gestión eficiente del recurso hídrico no es solo una cuestión ambiental, sino que tiene un impacto directo en la rentabilidad de la finca.

Una finca típica está subdividida en parcelas denominadas *tablones*. En este estudio, el cultivo es estrictamente caña de azúcar. Aunque se trata de un monocultivo, cada tablón alberga cepas en diferentes estadios de desarrollo (plantilla, soca de distintos cortes), lo que implica que cada unidad tiene necesidades hídricas y tolerancias al estrés hídrico distintas. El principal cuello de botella no suele ser la disponibilidad absoluta de agua, sino la infraestructura de distribución: la finca dispone de un único sistema de riego centralizado (como un pivote central o una cuadrilla de irrigación). Esta restricción física impone que los tablones deben atenderse de forma secuencial.

I-B. Restricciones Prácticas y Estrés Hídrico

El modelo agrónomo establece que la caña de azúcar tiene una "ventana ideal" para recibir irrigación. Iniciar el riego en este *tiempo óptimo* asegura el máximo rendimiento de biomasa y sacarosa. Desviarse de este punto, ya sea por adelanto o retraso, genera penalizaciones fisiológicas para el cultivo.

Si el retraso supera un límite denominado *tiempo de supervivencia*, el tablón entra en déficit hídrico agudo. En este punto la caña comienza a marchitarse y el costo penal, que representa las pérdidas económicas directas, crece en función de la prioridad comercial del tablón.

I-C. Motivación Tecnológica e Impacto

Históricamente, los ingenieros agrónomos han dependido de heurísticas intuitivas o programación manual para coordinar estos sistemas. Estas aproximaciones rara vez alcanzan el óptimo global, generando desperdicios económicos y un uso subóptimo del agua. La motivación de este proyecto es dotar al sector cañicultor de herramientas algorítmicas que calculen

el orden de riego que minimice la penalización total en la cosecha.

II. FORMALIZACIÓN MATEMÁTICA

II-A. Definición del Espacio del Problema: La Finca

Sea F el conjunto ordenado finito que representa la finca, compuesto por n tablones indexados de 0 a $n - 1$:

$$F = \langle T_0, T_1, \dots, T_{n-1} \rangle \quad (1)$$

Cada tablón T_i se representa mediante una tupla con cuatro parámetros:

$$T_i = \langle ts_i, tr_i, p_i, ro_i \rangle \quad (2)$$

- $ts_i \in \mathbb{N}$: **Tiempo de supervivencia**. Límite antes de empezar a perder valor.
- $tr_i \in \mathbb{N}^+$: **Tiempo de regado**. Duración de la tarea de riego.
- $p_i \in \{1, 2, 3, 4\}$: **Prioridad del tablón**. Factor de ponderación económica donde 4 es la mayor criticidad.
- $ro_i \in \mathbb{N}$: **Tiempo de riego óptimo**. El día ideal en el cual la penalidad es mínima.

Imponemos la restricción:

$$0 \leq ro_i \leq ts_i - tr_i \quad \forall i \in \{0, \dots, n-1\} \quad (3)$$

II-B. La Programación de Riego (Permutación)

Dado que el riego es secuencial, una solución al problema es una **permutación** Π del conjunto de índices $I = \{0, 1, \dots, n-1\}$.

$$\Pi = \langle \pi_0, \pi_1, \dots, \pi_{n-1} \rangle \quad (4)$$

Donde $\pi_k = i$ indica que el tablón T_i ocupará la k -ésima posición en la secuencia de riego.

II-C. Cálculo de Tiempos y Función de Costo Individual (CRF)

Para la primera posición, el tiempo de inicio es $t_{\pi_0}^{\Pi} = 0$. Para cualquier tablón subsiguiente π_j , su tiempo de inicio es el tiempo de finalización de su predecesor:

$$t_{\pi_j}^{\Pi} = t_{\pi_{j-1}}^{\Pi} + tr_{\pi_{j-1}} \quad (5)$$

La penalidad de un tablón i depende de t_i^{Π} . Se evalúan tres casos mutuamente excluyentes:

Caso 1 - Riego en Tiempo Óptimo:

$$CRF_{\Pi}[i] = ts_i - (t_i^{\Pi} + tr_i) \quad \text{si } t_i^{\Pi} = ro_i \quad (6)$$

Caso 2 - Retraso Tolerado sin Déficit Hídrico:

$$CRF_{\Pi}[i] = 2 \cdot (ts_i - (t_i^{\Pi} + tr_i)) \quad (7)$$

Caso 3 - Déficit Hídrico Crítico (Peor Escenario):

$$CRF_{\Pi}[i] = 2 \cdot p_i \cdot ((t_i^{\Pi} + tr_i) - ts_i) \quad (8)$$

II-D. Función de Costo Objetivo

El problema de optimización consiste en encontrar la permutación Π^* que minimice el costo total:

$$\Pi^* = \arg \min_{\Pi \in S_n} \left(\sum_{i=0}^{n-1} CRF_{\Pi}[i] \right) \quad (9)$$

III. NATURALEZA COMBINATORIA DEL PROBLEMA

El problema de encontrar la secuencia de riego óptima es un problema de secuenciamiento en una sola máquina [2], [3]. Dado que la función de penalidad es escalonada y depende de la posición en la secuencia, no tiene monotonía.

El dominio de soluciones equivale a todas las ordenaciones de los n tablonos, de modo que $|S_n| = n!$.

Cuadro I
CRECIMIENTO DE PERMUTACIONES VS TAMAÑO (n)

Tamaño (n)	Permutaciones ($n!$)	Tiempo aprox. a 1 μs
5	120	0.12 milisegundos
10	3,628,800	3.6 segundos
12	$4,79 \times 10^8$	~ 8 minutos
15	$1,30 \times 10^{12}$	~ 15 días
20	$2,43 \times 10^{18}$	$\sim 77,000$ años

Esta explosión factorial es la razón principal por la que el problema se clasifica como **NP-Hard** [1], y motiva el desarrollo de las estrategias algorítmicas que se presentan a continuación.

IV. ANÁLISIS COMPLETO DE EJEMPLOS Y FUNCIONAMIENTO INTERNO

Para ilustrar cómo opera cada algoritmo, analizamos el archivo `test1_in.txt` del proyecto: $n = 5$ con $T_0 = \langle 10, 10, 1, 0 \rangle$, $T_1 = \langle 11, 2, 1, 5 \rangle$, $T_2 = \langle 13, 5, 1, 7 \rangle$, $T_3 = \langle 9, 4, 4, 1 \rangle$ y $T_4 = \langle 14, 1, 2, 10 \rangle$.

Al ejecutar la implementación en Python, **los tres algoritmos coinciden en la misma solución**: Permutación $\Pi^* = \langle 2, 3, 1, 4, 0 \rangle$ con un costo mínimo de **44**. A continuación se detalla cómo cada enfoque llega a este resultado.

IV-A. Fuerza Bruta: El Árbol de Búsqueda (DFS)

La Fuerza Bruta genera un árbol recursivo explorando las $n! = 120$ combinaciones posibles. En la **Figura 1**, los caminos en rojo corresponden a decisiones subóptimas, mientras que el camino verde muestra la rama que alcanza el costo mínimo de 44.

El cálculo de la rama óptima es el siguiente:

- **Elige T_2 ($t = 0$):** Caso 2. $C = 2 \times (13 - (0 + 5)) = 16$. Nuevo $t = 5$. Acumulado = 16.
- **Elige T_3 ($t = 5$):** Caso 2. $C = 2 \times (9 - 9) = 0$. Nuevo $t = 9$. Acumulado = 16.
- **Elige T_1 ($t = 9$):** Caso 2. $C = 2 \times (11 - 11) = 0$. Nuevo $t = 11$. Acumulado = 16.
- **Elige T_4 ($t = 11$):** Caso 2. $C = 2 \times (14 - 12) = 4$. Nuevo $t = 12$. Acumulado = 20.
- **Elige T_0 ($t = 12$):** Marchitado (Caso 3). $C = 2 \times 1 \times (22 - 10) = 24$. Total = **44**.

IV-B. Voraz: Matriz de Descarte Secuencial y Método Piloto

El algoritmo Voraz no toma decisiones miopes; usa *Lookahead* y la heurística WMDD. En cada turno proyecta el impacto de cada decisión sobre los tablonos restantes.

En el **Turno 1** ($t = 0$), el algoritmo evalúa los 5 tablonos. Si elige T_0 primero (Costo Actual = 0), el costo simulado futuro es mayor a 80. Si elige T_2 primero (Costo Actual = 16), el costo futuro simulado baja a 28. Al comparar todas las proyecciones (ver **Figura 2**), elige T_2 porque minimiza el daño total proyectado ($16 + 28 = 44$). En el Turno 2 repite la lógica con los 4 tablonos restantes y elige T_3 , y así hasta completar la secuencia.

IV-C. Programación Dinámica: Matriz de Ensamblaje Bottom-Up

El diccionario de estados de la PD almacena subproblemas ya resueltos. A diferencia de la Fuerza Bruta, aquí la memoria es clave. La **Figura 3** muestra que cada entrada del diccionario responde a una sola pregunta: "¿Cuál es el costo mínimo para regar exactamente este subconjunto de tablonos?".

La resolución se construye de abajo hacia arriba: el algoritmo calcula que regar T_0 al final tiene un costo de 24. Con eso en memoria, el estado $\{T_0, T_4\}$ simplemente suma el costo local de T_4 (4) al costo guardado de $\{T_0\}$ (24), obteniendo 28. Esta cadena continúa hasta el estado inicial con los 5 tablonos, que confirma el óptimo global de 44. Si otra ruta necesita el costo de $\{T_0, T_4\}$, lo lee directamente del diccionario en $O(1)$, sin recalcular nada.

V. VALIDACIÓN EMPÍRICA

Los tres algoritmos se ejecutaron sobre 19 casos de prueba y produjeron costos equivalentes a la salida esperada oficial. El Test 10 (con 8 tablonos) merece atención particular: la salida esperada es 522, que la Fuerza Bruta y la PD confirman de forma exacta. El Voraz obtiene 532, lo que confirma su naturaleza de estrategia aproximada.

VI. ALGORITMO 1 — FUERZA BRUTA

VI-A. Idea Conceptual

La Fuerza Bruta construye todo el árbol de combinaciones posibles. En cada nivel de recursión elige un tablón no regado, calcula el costo de regarlo en el tiempo actual y baja recursivamente para resolver el resto de la finca.

VI-B. Complejidad Computacional (Fuerza Bruta)

Para n tablonos, el algoritmo evalúa todas las permutaciones posibles.

- **Complejidad Temporal ($O(n \cdot n!)$):** Sea $T(k)$ el tiempo para explorar un nodo con k tablonos pendientes. El algoritmo itera sobre los k elementos y en cada iteración construye la estructura `resto` en $O(k)$ y llama recursivamente a $T(k - 1)$:

$$T(k) = k \cdot (T(k - 1) + O(k)) \quad (10)$$

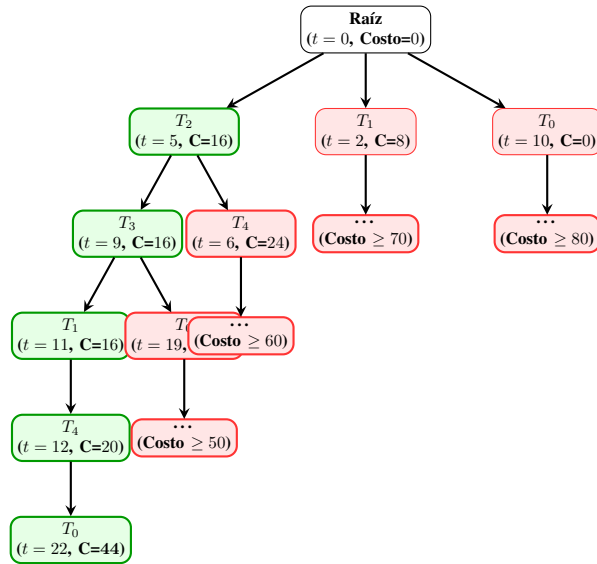


Figura 1. Visualización parcial del árbol de ejecución (Fuerza Bruta). El camino verde representa la decisión óptima Π^* y detalla el tiempo t y el costo acumulado (C) en cada nodo.

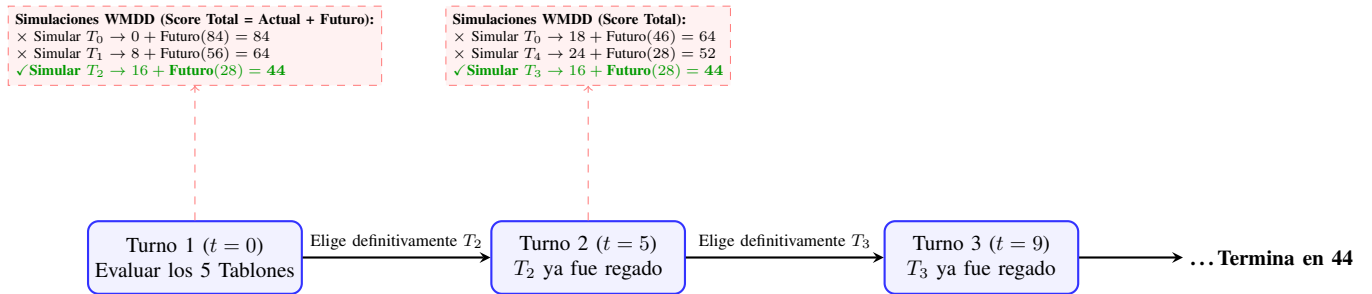


Figura 2. Línea de tiempo del algoritmo Voraz. El simulador WMDD calcula una proyección de penalidades y elige el camino que minimice el score total (costo local + costo futuro estimado).

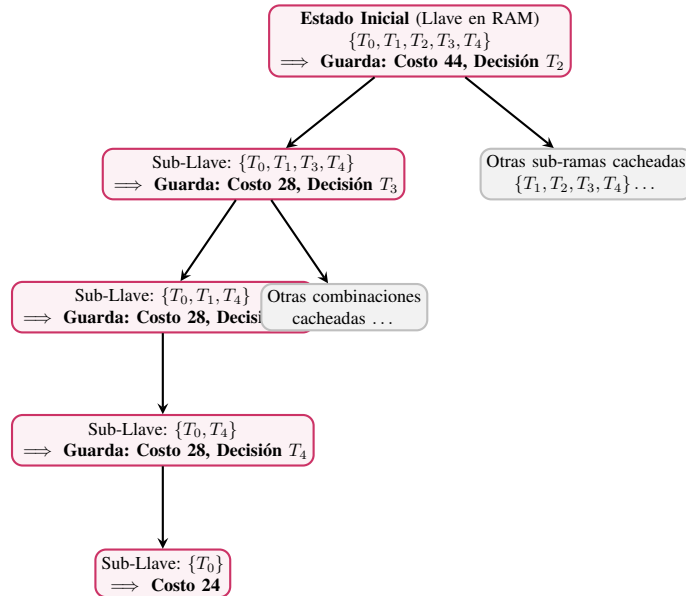


Figura 3. Grafo de dependencias de subproblemas (Programación Dinámica). Cada caja es una entrada en el diccionario de memoización. El algoritmo construye el costo óptimo global a partir de subproblemas resueltos previamente.

Algorithm 1 Fuerza Bruta: Encontrar Óptimo

```

1: procedure ExplorarSinMemo(pendientes, tiempo)
2: if pendientes =  $\emptyset$  then
3:   return 0, []
4: end if
5: mejor_costo  $\leftarrow \infty$ , mejor_orden  $\leftarrow []$ 
6: for idx  $\in$  pendientes do
7:   costo_local  $\leftarrow$  CostoTablon(idx, tiempo)
8:   resto  $\leftarrow$  pendientes  $\setminus$  {idx}
9:   t_nuevo  $\leftarrow$  tiempo + tr_idx
10:  costo_futuro, orden  $\leftarrow$ 
      ExplorarSinMemo(resto, t_nuevo)
11:  total  $\leftarrow$  costo_local + costo_futuro
12:  orden_candidata  $\leftarrow$  [idx]  $\cup$  orden
13:  if total < mejor_costo then
14:    mejor_costo  $\leftarrow$  total
15:    mejor_orden  $\leftarrow$  orden_candidata
16:  else if total = mejor_costo y orden_candidata >
      mejor_orden then
17:    mejor_costo  $\leftarrow$  total
18:    mejor_orden  $\leftarrow$  orden_candidata
19:  end if
20: end for
21: return mejor_costo, mejor_orden
22: end procedure

```

Desplegando la recurrencia:

$$\begin{aligned}
T(n) &= n \cdot T(n-1) + n \cdot O(n) \\
&= n(n-1)T(n-2) + n(n-1)O(n-1) + n \cdot O(n) \\
&= \sum_{i=0}^{n-1} \frac{n!}{(n-i)!} \cdot O(n-i)
\end{aligned}$$

El número total de nodos en el árbol de recursión está acotado por $\sum_{k=0}^n \frac{n!}{k!} \approx e \cdot n!$. Como cada nodo realiza $O(n)$ operaciones en el peor caso:

$$T(n) = O(n \cdot n!) \quad (11)$$

- **Complejidad Espacial ($O(n^2)$):** La ejecución es una búsqueda en profundidad (DFS) con profundidad máxima $h = n$. En cada nivel i se instancian arreglos temporales de tamaño $O(i)$. La memoria acumulada en cualquier instante es:

$$S(n) = \sum_{i=1}^n O(i) = O\left(\frac{n(n+1)}{2}\right) = O(n^2) \quad (12)$$

VI-C. Consistencia y Desempeño Lexicográfico

Un aspecto importante de la implementación es el manejo de empates: cuando dos permutaciones producen el mismo costo mínimo, los tres algoritmos deben elegir el mismo camino para poder verificar sus resultados entre sí. Para esto implementamos un **desempeño lexicográfico descendente**: si dos órdenes cuestan lo mismo, se elige el de índices mayores. Por ejemplo, si $[2, 1, 0]$ y $[0, 1, 2]$ tienen el mismo costo, el

algoritmo siempre elige $[2, 1, 0]$ porque empieza con 2, que es mayor que 0. Esto garantiza consistencia entre los tres algoritmos y simplifica la verificación de correctitud.

VI-D. Demostración Matemática (Fuerza Bruta)

La correctitud de la Fuerza Bruta se sigue directamente de que explora la totalidad del espacio de soluciones. Formalmente:

Teorema 1. El algoritmo de Fuerza Bruta garantiza encontrar el óptimo global Π^* .

Demostración. Sea S_n el espacio de permutaciones con $|S_n| = n!$, y $f : S_n \rightarrow \mathbb{R}^+$ la función de costo. El algoritmo genera el conjunto E de todas las evaluaciones realizadas. Por definición recursiva, $\forall \Pi \in S_n \implies \Pi \in E$, por lo tanto $E \equiv S_n$. El costo óptimo se define como $C^* = \min_{\Pi \in S_n} f(\Pi)$, y el algoritmo retorna $\hat{C} = \min_{\Pi \in E} f(\Pi)$. Como $E \equiv S_n$:

$$\hat{C} = \min_{\Pi \in E} f(\Pi) \equiv \min_{\Pi \in S_n} f(\Pi) \implies \hat{C} = C^* \quad (13)$$

□

VII. ALGORITMO 2 — VORAZ**VII-A. Heurística WMDD y Método Piloto**

El algoritmo Voraz evita la miopía clásica de estas técnicas usando un método de simulación *Lookahead* combinado con la heurística WMDD (Weighted Modified Due Date) [5].

En lugar de elegir el tablón más barato en el momento actual, el algoritmo simula el futuro: por cada candidato, finge haberlo regado ya y evalúa cómo quedarían los demás tablonos. Para ordenar ese "resto" rápidamente, usa la fórmula WMDD, que toma el tiempo de vida restante de cada tablón y lo divide por su prioridad económica. Si a un tablón de alta prioridad le queda poco tiempo, ese cociente es pequeño, lo que indica urgencia. El algoritmo suma el costo inmediato más el costo proyectado de la simulación y elige el candidato con el menor total. Así evita el error clásico de optimizar el paso actual a costa de penalizar los siguientes.

VII-B. Complejidad Computacional (Voraz con WMDD)

El lookahead introduce una anidación de simulaciones que aleja al algoritmo de las métricas de un voraz clásico.

- **Complejidad Temporal ($O(n^3 \log n)$):** En el turno con k tablonos pendientes, el algoritmo evalúa k candidatos. Para cada candidato aplica *timsort* sobre $k-1$ elementos ($O(k \log k)$) y luego recorre el resultado linealmente ($O(k)$). El trabajo por turno es:

$$W(k) = k \cdot [O(k \log k) + O(k)] \approx O(k^2 \log k) \quad (14)$$

El tiempo total suma el trabajo de todos los turnos:

$$T(n) = \sum_{k=1}^n O(k^2 \log k) \quad (15)$$

Algorithm 2 Algoritmo Voraz (Lookahead con WMDD)

```

1:  $orden \leftarrow \emptyset, tiempo \leftarrow 0, pendientes \leftarrow \{0..n-1\}$ 
2: while  $pendientes \neq \emptyset$  do
3:    $mejor\_candidato \leftarrow \text{NULO}, min\_proyeccion \leftarrow \infty$ 
4:   for  $c \in pendientes$  do
5:      $costo\_local \leftarrow \text{CostoTablon}(c, tiempo)$ 
6:      $t\_futuro \leftarrow tiempo + tr_c$ 
7:      $resto \leftarrow pendientes \setminus \{c\}$ 
8:      $resto\_ordenado \leftarrow \text{WMDD}(resto, t\_futuro)$ 
9:      $costo\_futuro \leftarrow \text{Simular}(resto\_ordenado, t\_futuro)$ 

10:     $puntaje \leftarrow costo\_local + costo\_futuro$ 
11:    if  $puntaje < min\_proyeccion$  then
12:       $min\_proyeccion \leftarrow puntaje$ 
13:       $mejor\_candidato \leftarrow c$ 
14:    else if  $puntaje = min\_proyeccion$  y  $c >$ 
       $mejor\_candidato$  then
15:       $mejor\_candidato \leftarrow c$ 
16:    end if
17:  end for
18:   $orden.push(mejor\_candidato)$ 
19:   $tiempo \leftarrow tiempo + tr_{mejor\_candidato}$ 
20:   $pendientes \leftarrow pendientes \setminus \{mejor\_candidato\}$ 
21: end while
22: return  $orden$ 

```

Acotando $\log k \leq \log n$ como constante y usando la identidad de la suma de cuadrados:

$$T(n) \leq \log n \left(\sum_{k=1}^n k^2 \right) = \log n \cdot \frac{n(n+1)(2n+1)}{6} \quad (16)$$

$$T(n) = O(n^3 \log n) \quad (17)$$

- **Complejidad Espacial ($O(n)$):** El algoritmo mantiene solo colecciones de tamaño $\leq n$ en todo momento. No hay almacenamiento exponencial ni recursión profunda:

$$E(n) = O(n) + O(n) + O(n-1) = O(n) \quad (18)$$

Esto le permite procesar fincas grandes sin consumo excesivo de memoria.

VII-C. Limitaciones Teóricas

Al tomar decisiones locales sin explorar todas las permutaciones, el Voraz no garantiza el óptimo global. En casos como el Test 10, el error es de aproximadamente 1,9%. Sin embargo, produce resultados muy cercanos al óptimo sin volverse inviable computacionalmente.

VII-D. Demostración Matemática (Algoritmo Voraz)

En cada turno el Voraz toma la mejor decisión disponible según su estimación del futuro, pero esa estimación puede diferir del valor real. Esto es lo que formalmente lo hace subóptimo a nivel global:

Teorema 2. *El algoritmo Voraz garantiza el óptimo local en cada paso t , pero puede ser globalmente subóptimo.*

Demostración. Sea P_t el conjunto de tablonos disponibles en el instante t . Sea $f_{loc} : P_t \times \mathbb{R} \rightarrow \mathbb{R}$ el costo unitario, y $f_{sim} : \mathcal{P}(P_t) \times \mathbb{R} \rightarrow \mathbb{R}$ la función heurística WMDD de proyección. El algoritmo elige c^* mediante:

$$c^* = \arg \min_{c \in P_t} [f_{loc}(c, t) + f_{sim}(P_t \setminus \{c\}, t + tr_c)] \quad (19)$$

Esta selección garantiza el óptimo condicionado al estado actual. Sin embargo, dado el carácter heurístico de WMDD, existe $P_t \subset F$ tal que $f_{sim}(P_t, t) \neq f_{opt}(P_t, t)$, donde f_{opt} es el costo real futuro. Acumulando el error sobre los n pasos:

$$C_V \geq \sum_{k=1}^n f_{loc}(\pi_k^*, t_k) \implies C_V \geq C^* \quad (20)$$

□

VIII. ALGORITMO 3 — PROGRAMACIÓN DINÁMICA**VIII-A. Subestructura Óptima y Memoización**

La Programación Dinámica (PD) supera a la Fuerza Bruta almacenando resultados de subproblemas ya resueltos. El estado de la finca depende únicamente de *qué tablonos quedan por regar*, sin importar el orden histórico en que se regaron los anteriores.

Sea $C(P, t)$ el costo óptimo para los tablonos pendientes en P al tiempo t :

$$C(P, t) = \min_{i \in P} [\text{Costo}(i, t) + C(P \setminus \{i\}, t + tr_i)] \quad (21)$$

Si el algoritmo llega a un conjunto P que ya fue resuelto antes, simplemente devuelve su valor del diccionario en $O(1)$.

La diferencia con la Fuerza Bruta es que la PD no recalcula subproblemas. Si ya sabemos cuál es el costo mínimo para regar exactamente los tablonos 3 y 4 a partir del día 10, ese resultado queda guardado. La próxima vez que por cualquier ruta lleguemos al día 10 con los tablonos 3 y 4 pendientes, no se ejecuta ningún cálculo extra: se lee directamente del diccionario. Esto elimina millones de cálculos redundantes.

VIII-B. Complejidad Computacional (Programación Dinámica)

La memoización reduce el crecimiento factorial a uno exponencial, al evitar que los mismos subproblemas se calculen más de una vez.

- **Complejidad Temporal ($O(n \cdot 2^n)$):** Para n tablonos existen 2^n subconjuntos posibles. Gracias al diccionario con acceso $O(1)$, ningún subconjunto se evalúa dos veces. Cuando el algoritmo procesa un subconjunto de tamaño k , itera k veces para aplicar el operador mín. El tiempo total es:

$$T(n) = \sum_{k=1}^n \binom{n}{k} \cdot O(k) \quad (22)$$

Algorithm 3 Programación Dinámica Top-Down

```
1: Global: Diccionarios memo_valor y memo_decision
2: procedure CalcularValorOptimo(pendientes, tiempo)
3: if pendientes =  $\emptyset$  then
4:   return 0
5: end if
6: if pendientes  $\in$  memo_valor then
7:   return memo_valor[pendientes]
8: end if
9: mejor_costo  $\leftarrow \infty$ , mejor_idx  $\leftarrow -1$ 
10: for idx  $\in$  pendientes do
11:   costo_local  $\leftarrow$  CostoTablon(idx, tiempo)
12:   resto  $\leftarrow$  pendientes  $\setminus$  {idx}
13:   costo_resto  $\leftarrow$  CalcularValorOptimo(resto, tiempo + tr_idx)
14:   total  $\leftarrow$  costo_local + costo_resto
15:   if total < mejor_costo then
16:     mejor_costo  $\leftarrow$  total
17:     mejor_idx  $\leftarrow$  idx
18:   else if total = mejor_costo y idx > mejor_idx then
19:     mejor_idx  $\leftarrow$  idx
20:   end if
21: end for
22: memo_valor[pendientes]  $\leftarrow$  mejor_costo
23: memo_decision[pendientes]  $\leftarrow$  mejor_idx
24: return mejor_costo
25: end procedure
```

Aplicando la identidad combinatoria $k \binom{n}{k} = n \binom{n-1}{k-1}$:

$$\begin{aligned} T(n) &= O\left(\sum_{k=1}^n n \binom{n-1}{k-1}\right) \\ &= O\left(n \sum_{j=0}^{n-1} \binom{n-1}{j}\right) \end{aligned}$$

Por el Teorema del Binomio de Newton $(1+1)^{n-1} = 2^{n-1}$:

$$T(n) = O(n \cdot 2^{n-1}) \equiv O(n \cdot 2^n) \quad (23)$$

- **Complejidad Espacial** ($O(n \cdot 2^n)$): Los diccionarios deben almacenar los resultados de los 2^n subproblemas posibles. Dado que cada entrada tiene un tamaño proporcional a k , la memoria total requerida es:

$$S(n) = \sum_{k=1}^n \binom{n}{k} \cdot O(k) = O(n \cdot 2^n) \quad (24)$$

Este crecimiento exponencial en memoria explica por qué el algoritmo agota la RAM al superar $n \approx 25$: 2^{25} entradas son aproximadamente 33 millones de registros en el diccionario.

VIII-C. Demostración Matemática (Programación Dinámica)

La correctitud de la PD se basa en el principio de optimalidad de Bellman: si la solución a un subproblema es óptima, entonces la solución global que la incluye también lo es. Por inducción fuerte sobre el tamaño del conjunto pendiente, se puede demostrar que cada entrada del diccionario contiene el costo mínimo real para ese subconjunto.

Teorema 3. La Programación Dinámica garantiza el óptimo global C^* .

Demostración. Procedemos por inducción fuerte sobre $k = |P|$.

Base ($k = 0$): Si $P = \emptyset$, no hay tableros pendientes y $C(\emptyset, t) = 0$, que es trivialmente óptimo.

Hipótesis inductiva ($k-1$): Para todo P' con $|P'| = k-1$, asumimos que $C(P', t') = C_{opt}^*(P', t')$.

Paso inductivo (k): Sea $|P| = k$. Por la ecuación de Bellman:

$$C(P, t) = \min_{i \in P} [f(i, t) + C(P \setminus \{i\}, t + tr_i)] \quad (25)$$

Para todo $i \in P$, el conjunto $P \setminus \{i\}$ tiene tamaño $k-1$. Por la hipótesis inductiva, $C(P \setminus \{i\}, \cdot) = C_{opt}^*$. Como $f(i, t)$ es un costo determinista exacto y el operador mín se evalúa sobre todos los $i \in P$:

$$C(P, t) = \min_{i \in P} [f(i, t) + C_{opt}^*(P \setminus \{i\}, t + tr_i)] \equiv C^* \quad (26)$$

Por inducción fuerte, la propiedad se cumple para todo $k \leq n$. $\square$

IX. DISEÑO DE CASOS DE PRUEBA

Los 19 casos de prueba cubren escenarios variados:

- Casos pequeños ($n = 5$) donde el déficit hídrico es bajo y fácil de manejar.
- Escenarios con prioridades mixtas, donde elegir primero tableros rápidos perjudica a tableros de alta prioridad ($p_i = 4$).
- Distribuciones complejas con empates teóricos, como el Test 10, resueltos por la regla lexicográfica.
- Instancias de escala mayor ($n = 20$ y $n = 25$) para evaluar los límites de cada algoritmo.

La estructura valida la exactitud de la Fuerza Bruta y la PD, y permite medir el margen de error admisible del Voraz.

X. COMPARACIÓN GLOBAL DE LOS ALGORITMOS

La Programación Dinámica produce resultados exactos hasta $n \approx 18$ (230 ms para $n = 15$). A partir de $n \geq 20$ el consumo de memoria se vuelve prohibitivo. El Voraz resuelve el caso $n = 25$ en 3 milisegundos con variaciones menores al 3%. La Fuerza Bruta, aunque exacta, no es viable en la práctica para $n \geq 12$.

XI. DETALLES DE IMPLEMENTACIÓN

La arquitectura del sistema separa claramente las responsabilidades. Un **Frontend** moderno gestiona la interfaz, y un **Backend** en Node.js expone endpoints REST que reciben las fincas en formato JSON. El motor algorítmico está codificado en **Python** e invocado mediante pipes desde el backend. Los módulos están organizados por función: `modelo_riego.py` contiene los tres algoritmos, `costo_tablon.py` implementa la función de penalidad y `utils_estado.py` maneja la representación del estado de tablonos pendientes.

XII. ANÁLISIS DE CORRECTITUD

El principio de optimalidad de Bellman [4] fundamenta la correctitud de la PD: un camino óptimo está compuesto por sub-caminos que también son óptimos. Al visitar cada subconjunto de tablonos de forma top-down con memoización, cada resultado guardado es globalmente correcto. Por su parte, la Fuerza Bruta garantiza correctitud por exhaustividad: evalúa el 100 % de las permutaciones, por lo que no puede dejar el óptimo sin evaluar.

XIII. RESULTADOS EXPERIMENTALES

Cuadro II
COSTOS Y RENDIMIENTO (MUESTRA ALEATORIA DE PRUEBAS)

Test	n	Costo Ideal	Costo V	T. FB (ms)	T. PD (ms)
1	5	44	44	0	0
8	7	294	294	12	0
10	8	522	532	92	0
12	10	526	542	8214	4
15	15	1030	1037	Timeout	230
19	25	3594	3782	Timeout	Timeout

A medida que n aumenta, el Voraz presenta un margen de error creciente pero contenido. El tiempo de la Fuerza Bruta crece un factor de $\sim n$ por cada tablón adicional (de 92 ms a 8 segundos al pasar de $n = 8$ a $n = 10$). El uso de memoria de la PD se convierte en el factor limitante en los últimos tests.

XIII-A. Análisis Detallado del Comportamiento por Algoritmo

Entradas pequeñas (Test 1 y Test 8): Para $n = 5$ y $n = 7$, los tres algoritmos producen el mismo resultado (44 y 294). Con 120 y 5.040 combinaciones respectivamente, la Fuerza Bruta y la PD evalúan todo el espacio en tiempo despreciable. El Voraz también acierta porque con horizontes cortos su estimación WMDD coincide con el óptimo real.

El punto de divergencia (Test 10, $n = 8$): La Fuerza Bruta y la PD coinciden en **522**, evaluando las 40.320 combinaciones posibles. El Voraz obtiene **532**, fallando por 10 puntos. La causa es la *miopía heurística*: en un turno intermedio, el Voraz evaluó que postergar un tablón A de alta prioridad causaría un alto daño futuro, y lo regó de inmediato. Sin embargo, existía una combinación posterior que permitía acomodar A sin penalidades. La PD y la Fuerza Bruta encontraron esa

combinación porque exploraron el espacio completo; el Voraz no, porque solo trabajó con una estimación.

El límite de escala (Test 15 y Test 19): Para $n = 15$, la Fuerza Bruta supera el tiempo límite: 1,3 billones de permutaciones exceden la capacidad de cómputo disponible. La PD resuelve el mismo caso en 230 ms encontrando el óptimo **1030**. En $n = 25$ la PD también falla, esta vez por memoria: 2^{25} subconjuntos generan cerca de 33 millones de entradas en el diccionario. El Voraz resuelve este caso en milisegundos con un costo de **3782**, lo que lo convierte en la única opción viable para fincas grandes.

XIV. DISCUSIÓN ACADÉMICA

En un contexto agroindustrial real, no siempre vale la pena buscar el óptimo exacto si calcularlo requiere tiempos de cómputo inviables. El Voraz, basado en heurísticas con respaldo teórico del área de Scheduling, produce soluciones con un error de entre 1 % y 3 % que se calculan en milisegundos, incluso en hardware de gama baja. Este balance entre calidad de solución y tiempo de respuesta lo hace más adecuado para escenarios de producción reales que los algoritmos exactos.

XV. TRABAJOS FUTUROS

Para instancias con $n > 20$ donde se requiera mayor exactitud, una dirección natural es explorar metaheurísticas como Algoritmos Genéticos o Simulated Annealing. También se identifican dos optimizaciones concretas sobre los algoritmos actuales:

- **Poda (Branch and Bound):** En la Fuerza Bruta, mantener el mejor costo parcial encontrado y descartar ramas que ya lo superan, reduciendo el número de evaluaciones sin sacrificar exactitud.
- **Paralelización del lookahead:** En el Voraz, el ciclo de simulaciones de candidatos puede ejecutarse en paralelo con procesamiento multi-core, reduciendo el tiempo total en fincas grandes.

XVI. CONCLUSIONES

El problema de secuenciamiento de riego con penalizaciones por tiempo no tiene solución exacta en tiempo polinomial dado su carácter NP-Hard. La Programación Dinámica reduce la complejidad de $O(n!)$ a $O(n \cdot 2^n)$, lo que la hace viable hasta $n \approx 18$. Para fincas más grandes, el Voraz con lookahead predictivo resulta ser la alternativa más práctica: escala bien, consume poca memoria y produce resultados con márgenes de error aceptables para el contexto agroindustrial.

XVII. REFERENCIAS

REFERENCIAS

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, y C. Stein, *Introduction to Algorithms*, 3ra ed. Cambridge, MA: MIT Press, 2009. <https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/>
- [2] R. L. Graham, E. L. Lawler, J. K. Lenstra, y A. H. G. Rinnooy Kan, "Optimization and Approximation in Deterministic Sequencing and Scheduling: A Survey," *Annals of Discrete Mathematics*, vol. 5, pp. 287–326, 1979. DOI: 10.1016/S0167-5060(08)70356-X

- [3] M. L. Pinedo, *Scheduling: Theory, Algorithms, and Systems*, 5ta ed. Nueva York, NY: Springer, 2016.
DOI: 10.1007/978-3-319-26580-3
- [4] R. Bellman, "Dynamic Programming," *Science*, vol. 153, no. 3731, pp. 34–37, 1966.
DOI: 10.1126/science.153.3731.34
- [5] T. E. Morton y D. W. Pentico, *Heuristic Scheduling Systems*. Nueva York, NY: Wiley, 1993.
DOI: 10.1002/9781118627372